

Architecture of a Fully Automated Fiat-to-Crypto TWAP Buyback Mechanism for Corporate Treasuries on the Polygon Network

The Convergence of Web2 Revenue and Web3 Tokenomics

The structural integration of traditional Web2 affiliate revenue streams with decentralized Web3 tokenomic models represents a highly complex infrastructure challenge. For digital travel enterprises, the generation of fiat revenue via platforms such as Booking.com and Viator provides a reliable, albeit asynchronous, cash flow. Strategically redirecting this capital to programmatically purchase a native ERC-20 utility token (\$FARA) on decentralized exchanges (DEXs) serves as a potent mechanism for establishing continuous market support, accumulating protocol-owned liquidity, and executing deflationary burn dynamics.

However, constructing this automated fiat-to-crypto pipeline requires navigating a labyrinth of regulatory gateways, asynchronous settlement timelines, on-chain custody constraints, and adversarial execution environments. A naive approach—such as manually bridging fiat to a centralized exchange hot wallet, or instantly deploying market buy orders on an Automated Market Maker (AMM)—inevitably leads to catastrophic single points of failure, severe price slippage, and value extraction by predatory Maximal Extractable Value (MEV) bots.

This comprehensive report architects a robust, fully automated, and cryptographically secure treasury pipeline. By leveraging the Polygon Proof-of-Stake (PoS) network for execution, Circle Mint for fiat ingestion, Gnosis Safe Zodiac modules for autonomous custody, 1inch Fusion for intent-based order routing, and Chainlink Automation for decentralized orchestration, enterprise treasuries can execute Time-Weighted Average Price (TWAP) buybacks with mathematical precision and zero operational overhead.

Affiliate Revenue Ingestion and Treasury Buffering Dynamics

The foundational layer of the buyback mechanism involves capturing fiat revenue from Web2 travel affiliate networks. To design a mathematically sound TWAP execution algorithm, the infrastructure must first reconcile the severe disparities in payout chronologies, commission structures, and settlement thresholds inherent in these traditional networks.

Asynchronous Cash Flow Constraints

The enterprise generates revenue through two primary affiliate programs, each presenting distinct treasury management challenges:

1. **The Viator Partner Program:** Viator operates on a standard 8% commission model for any travel experience booked through the platform, utilizing a 30-day cookie tracking window for standard affiliate links and up to a 90-day window for Travel Agents Share With Client links.¹ The platform supports multiple integration topologies, ranging from basic iFrame embeds to full custom checkout experiences via the Viator Affiliate API, which supports multi-vendor carts and 3DS compliance.¹ Crucially for treasury velocity, Viator processes disbursements on a weekly cadence, typically routing funds via PayPal or traditional bank transfers.²
2. **The Booking.com Affiliate Program:** In stark contrast, Booking.com implements a deeply lagged, tiered commission structure predicated on stayed reservations.⁴ Revenue generation is subjected to a two-month validation delay. For example, commissions for reservations completed in April are calculated and disbursed within the first five working days of June.⁴ Furthermore, Booking.com enforces a strict 100 EUR minimum payment threshold. If the accumulated commission falls below this boundary, the balance rolls over into subsequent months until the threshold is breached.⁴ Payouts are executed via Direct Bank Transfer in standardized fiat currencies (EUR, USD, GBP).⁴

The Time-Weighted Average Price (TWAP) Imperative

The stark dichotomy between Viator's weekly disbursements and Booking.com's delayed, batch-oriented monthly payouts creates a highly irregular, burst-like fiat influx. If the corporate treasury were to instantly convert a consolidated monthly deposit into USDC and execute a single monolithic buy order for the \$FARA token, the resulting price impact would be highly detrimental. Concentrated liquidity shocks generate temporary price spikes, allowing arbitrageurs to extract value as the price reverts to its mean, effectively eroding the purchasing power of the treasury's capital.

To mitigate this systemic inefficiency, the architecture must implement a decoupled buffering phase. The asynchronous fiat deposits must be swept into a unified digital vault and algorithmic dispersed into the decentralized market via a TWAP strategy. By mathematically fractionating the aggregated monthly affiliate revenue into micro-purchases executed hourly or daily, the protocol absorbs the irregular fiat influx and applies continuous, predictable buy pressure to the \$FARA token.⁵ This temporal distribution minimizes slippage, masks institutional accumulation from predatory front-runners, and ensures optimal capital deployment regardless of Web2 payout delays.

Fiat-to-Crypto API Gateway Architecture

Transitioning capital from the legacy banking system to the Polygon blockchain requires a

regulated, programmable gateway capable of programmatic API interaction. The selection of this infrastructure dictates the regulatory compliance burden, the settlement latency, and the economic efficiency of the buyback program.

Gateway Evaluation: Stripe Crypto Onramp vs. Circle Mint

The enterprise market currently offers two dominant infrastructural solutions for bridging fiat to crypto: Stripe Crypto Onramp and the Circle Mint API.

Architectural Dimension	Stripe Crypto Onramp	Circle Mint Core API
Primary Target Architecture	Consumer-facing B2C retail purchases directly embedded within dApps or checkouts. ⁶	Institutional B2B treasury management, high-volume automated minting, and enterprise payouts. ⁸
Merchant of Record & Liability	Stripe acts as the merchant of record, assuming full liability for fraud and disputes. ⁷	The corporate entity maintains sovereign control over its own banking relationships and accounts. ⁹
KYC/AML Regulatory Burden	Stripe enforces per-user KYC, biometric validation, and real-time sanctions screening during the checkout flow. ⁷	The enterprise undergoes a stringent institutional KYC/KYB background check once during account creation. ¹¹
Economic Efficiency & Fees	Imposes processing fees and dynamic foreign exchange (FX) spread markups, which can degrade unit economics. ¹²	Guarantees 1:1 parity for USDC minting with zero hidden spread markups on USD deposits. ⁹
Execution Path & Inventory	Acts as an aggregator, routing orders across external exchanges or market makers using price-aware logic. ¹⁰	Issues native stablecoins directly from Circle, the sole regulated issuer of USDC and EURC. ⁹

A rigorous analysis of these parameters reveals that Stripe Crypto Onramp is structurally misaligned with the requirements of a corporate treasury buyback mechanism. Stripe is explicitly engineered to shield application developers from the immense regulatory overhead associated with individual retail consumer transactions, utilizing automated KYC tools to detect

synthetic IDs and recycled credentials.⁷ Applying this retail-oriented infrastructure to a closed-loop corporate revenue sweep would force the treasury to absorb unnecessary processing fees and opaque spread markups.¹²

Conversely, Circle Mint emerges as the optimal institutional gateway. It is specifically restricted to institutional customers—such as exchanges, wallet providers, and corporate treasuries—and is completely unavailable to retail individuals.¹¹ Circle Mint enables businesses to deposit traditional fiat from over 200 countries and instantly, programmatically convert it to USDC at a strict 1:1 ratio.⁸

Plaid Integration and Automated ACH Sweeps

To achieve end-to-end automation without human intervention, the fiat ingestion pipeline integrates Circle's Core Functionality API with Plaid's account verification network. This integration eliminates manual wire transfers and standardizes the funding mechanism.

1. **Secure Account Linking:** Plaid's client-side integration authenticates the traditional corporate bank account that receives the Booking.com and Viator deposits.¹³ By utilizing Plaid's Auth product, the enterprise verifies the external account instantly without storing highly sensitive banking credentials locally. Plaid generates a secure, unique `processor_token` corresponding to the verified bank account.¹³
2. **API Authorization:** The enterprise's backend Node.js infrastructure utilizes this `processor_token` to securely link the bank account to the Circle Mint Account. This is executed via a POST request to the `/v1/businessAccount/banks/wires` or ACH endpoints within the Circle API, passing the authentication token via the Bearer header.¹³
3. **Programmatic Sweeps:** Once the linkage is established, the backend can issue automated API requests to initiate ACH pulls from the traditional bank account.¹⁴ Circle's integration with Plaid dramatically streamlines the fiat-to-crypto operations, enabling a fast, fully automated journey into USDC with reduced operational overhead and accelerated settlement times.¹⁴
4. **Event-Driven Webhook Confirmations:** Efficient infrastructure relies on event-driven architecture rather than inefficient continuous polling. Circle provides robust webhook subscription management through its developer console.¹⁸ The backend subscribes to specific webhook notifications, allowing Circle to send an automated HTTP payload to the application the precise moment a fiat transaction has settled and the USDC has been successfully minted.⁸

Bridging Liquidity to the Polygon Network

Once the fiat has settled and USDC is minted within the Circle architecture, the capital must be securely transferred to the on-chain execution environment. The Polygon PoS network is selected as the primary settlement layer due to its multi-layered architecture, uncompromising performance, and low transaction costs, making it the industry default for high-volume,

micro-transaction USDC payments.¹⁹

To execute this transfer programmatically, the system utilizes Circle Mint's Crypto Payouts API.²¹ The Node.js backend first constructs a POST request to `/v1/addressBook/recipients` to explicitly whitelist the corporate Polygon treasury wallet address.²² If delayed withdrawals are disabled in the Circle Mint console, the recipient becomes active within seconds.²²

Crucially, because Circle is the native issuer of USDC, it issues the token natively on Polygon. The API routes the funds directly to the Polygon native USDC contract address (`0x3c499c542cEF5E3811e1192ce70d8cC03d5c3359`), completely circumventing the need for risky, third-party cross-chain bridges (such as the deprecated bridged USDC.e).²⁰ This direct issuance eliminates bridge vulnerability vectors and reduces settlement time to mere seconds.²⁴

On-Chain Custody, Security Primitives, and Multisig Automation

Before deploying accumulated capital into decentralized execution venues, the architecture must establish an impenetrable custody environment on the Polygon network. The reliance on standard Externally Owned Accounts (EOAs) or centralized hot wallets to hold treasury funds and execute automated buybacks introduces a catastrophic vulnerability: if the server hosting the automation script is compromised, the private keys controlling the treasury are entirely exposed, leading to total asset depletion.²⁵

Safe (Gnosis) Multisignature Architecture

The definitive standard for institutional digital asset custody is the Safe (formerly Gnosis Safe) multisignature wallet framework.²⁷ A multisig wallet enforces a structural distribution of signing authority across multiple independent cryptographic keys, utilizing an M-of-N threshold model (e.g., 2-of-3, or 3-of-5).²⁸ This completely eradicates single points of failure. Transactions require coordinated approval, ensuring no single individual, compromised server, or rogue script can unilaterally move corporate funds.²⁸

Operating an enterprise multisig requires stringent security hygiene. Hardware wallets must be sourced directly from manufacturers, and the private keys used for signing must never have been exposed to a hot wallet environment.²⁵ Furthermore, high-value transaction signing should occur on dedicated, potentially air-gapped workstations, utilizing block explorers to decode transaction payloads manually before signing to prevent UI spoofing attacks.²⁵ The infamous Bybit hack highlighted the critical necessity of disabling `delegatecall` functionality unless absolutely necessary, and verifying the software supply chain to prevent attackers from altering wallet contract logic.²⁶

The Paradox of Automation and Zodiac Modules

While multisig architecture provides unparalleled security, it presents a functional paradox for automated buyback operations: if a transaction strictly requires multiple human beings to physically sign a hardware wallet prompt, how can a programmatic Node.js script execute highly frequent TWAP buybacks every hour?

The architectural resolution to this paradox is the implementation of Safe Modules.²⁷ Modules are highly secure, explicit smart contract extensions that Safe owners can authorize.²⁷ Once enabled, a module has the unlimited ability to execute transactions from the Safe's context, spending Safe funds without requiring M-of-N hardware signatures for every single execution.²⁷

To facilitate the automated \$FARA buyback while maintaining strict security, the enterprise deploys a specialized StipendModule or VestingModule pattern.²⁷

1. The multisig owners deploy a customized module contract and explicitly whitelist it within the Safe interface.²⁷
2. This module contains immutable, highly constrained logic. It is programmed to only permit the transfer of a specific maximum amount of USDC at predefined intervals.
3. Crucially, the module hardcodes the destination routing. It forces the USDC into a specific DEX router contract and mandates that the output \$FARA tokens be sent directly to a verifiable burn address or a time-locked treasury vault.³²
4. An automated script (or decentralized oracle network) can call the executeBuyback() function on this module at regular intervals.²⁷

By implementing this architecture, the protocol achieves absolute separation of execution authorization and asset custody. Even if the Node.js automation script is fully compromised and the attacker seizes the automated caller's private key, the damage is fundamentally contained. The attacker can only trigger the buyback of \$FARA tokens prematurely; they are mathematically prevented from altering the recipient address, draining the USDC to an external wallet, or rewriting the underlying logic of the contract.²⁷

OpenZeppelin Contract Standards and Defender Deprecation

In the final stage of the buyback, the acquired \$FARA tokens must be processed according to the enterprise's tokenomic strategy. Rather than writing custom, unvetted Solidity logic, the treasury should utilize OpenZeppelin's standard library of secure, community-vetted modular smart contracts.³² Specifically, implementing the ERC20Burnable extension provides a standardized, permissioned function to permanently remove the bought-back \$FARA tokens from the total circulating supply.³² This should be combined with AccessControl and Pausable extensions to restrict burning authority and provide an emergency halt switch during anomalous market conditions.³²

Historically, enterprise teams utilized OpenZeppelin Defender—a mission-critical developer platform—to automate on-chain operations, schedule jobs, and monitor treasury management.³⁵ However, it is critical to note that OpenZeppelin disabled new sign-ups for

Defender on June 30, 2025, and announced a final sunset of the platform scheduled for July 1, 2026.³⁵ Consequently, building new automation infrastructure reliant on Defender's legacy hosted services represents a severe technical debt risk. The architecture must instead rely on self-hosted OpenZeppelin Relay configurations³³ or decentralized oracle networks for future-proof automation.

Execution Venues: AMMs vs. Intent-Based Aggregation

The core objective of the protocol is to swap the accumulated USDC for the native \$FARA token efficiently. The execution venue and the algorithmic parameters governing the trade dictate the efficiency of the capital deployment. In a fragmented, multi-venue crypto market, execution quality—encompassing slippage, liquidity access, and latency—often dictates the success of a TWAP strategy.³⁸ The architecture evaluates two primary decentralized execution models: Automated Market Makers (Uniswap V3) and Intent-Based Aggregation (1inch Fusion).

Option A: Algorithmic Execution via Uniswap V3

Uniswap V3 revolutionized decentralized trading through the introduction of concentrated liquidity, allowing liquidity providers to allocate capital within highly specific price ranges.³⁹ While exceptionally capital-efficient for stablecoin pairs, concentrated liquidity creates severe slippage vulnerabilities for volatile, long-tail tokens like \$FARA. If a market buy order exceeds the depth of the actively populated liquidity tick, the price impact accelerates rapidly as the AMM algorithm sweeps through sparse order book ranges to fulfill the trade.⁴⁰

When automating swaps via Node.js using the ethers.js library⁴¹, the developer must interface directly with the Uniswap V3 SwapRouter contract.⁴⁴ To mitigate catastrophic slippage programmatically, the automation script must construct the transaction utilizing the `sqrtPriceLimitX96` parameter within the swap function payload.⁴⁵ This parameter establishes a hard mathematical boundary for the execution price.⁴⁵

In the Uniswap V3 architecture, the price boundary is enforced as follows:

If `zeroForOne = true`, execution halts if `currentPrice < sqrtPriceLimitX96`

If `zeroForOne = false`, execution halts if `currentPrice > sqrtPriceLimitX96`

If the TWAP automation script triggers a swap and the resulting market impact pushes the price beyond the calculated `sqrtPriceLimitX96`, the transaction will revert with an `InvalidPriceLimit()` error.⁴⁵ While this mechanism successfully protects the corporate treasury from massive slippage, it introduces significant operational friction. Reverted transactions still consume base layer gas fees (MATIC/POL)⁴⁵, and the treasury fails to execute its intended buyback for that specific TWAP epoch, requiring complex retry logic and error handling.⁴⁷

Option B: Intent-Based Execution via 1inch Fusion (Recommended)

To resolve the slippage constraints and gas inefficiencies of direct AMM interaction, the architecture leverages the 1inch Fusion API.⁴⁹ 1inch Fusion operates on an intent-based architecture, representing a paradigm shift from traditional transaction broadcasting.⁵¹

Instead of generating a hardcoded transaction that interacts directly with a specific liquidity pool, the Safe module generates an off-chain "intent" or limit order.⁵¹ This intent explicitly states the minimum acceptable amount of \$FARA expected for a specific amount of USDC.

This system relies on a decentralized, competitive network of professional market makers known as "resolvers".⁵¹ Resolvers continuously monitor the 1inch Limit Order Protocol and compete to fill the user's intent via a Dutch auction mechanism.⁵² To participate, resolvers must undergo verification, stake sufficient 1INCH tokens to gain "Unicorn Power," rank among the top 10 registered resolvers, and deposit tokens into a FeeBank to cover resolving fees.⁵⁴

Architectural Dimension	Uniswap V3 Direct Execution	1inch Fusion Intent Execution
Routing Mechanism	Hardcoded path through specific smart contract pools. ³⁹	Resolvers source liquidity globally across CEXs, multiple DEXs, and PMMs to find the best rate. ⁵⁵
Gas Fee Economics	Paid by the corporate treasury initiating the swap.	Paid entirely by the resolver filling the order. ⁵³ Zero gas cost for the treasury. ⁵⁶
Slippage Protection	Requires complex, manual <code>sqrtPriceLimitX96</code> calculation off-chain prior to execution. ⁴⁵	Built-in via strict minimum return constraints; the resolver absorbs all execution and price risk. ⁵¹
Failed Transaction Cost	Reverts consume MATIC/POL gas fees, degrading capital efficiency. ⁴⁶	Absolutely no cost for expired, failed, or unfilled intents. ⁵³

The Dutch auction mechanism is highly beneficial for corporate buybacks. When the intent is published, the auction starts at a premium rate and gradually decays over a predefined duration. The 1inch Fusion SDK allows the enterprise to configure exact presets⁵⁶:

- **Auto:** Automatically adjusts the rate and expiration based on real-time market

conditions.⁵⁶

- **Fast:** Sets a rapid 3-minute auction for immediate execution, prioritizing speed over rate optimization.⁵⁶
- **Fair:** Extends the auction duration to 6 minutes, allowing resolvers more time to find a favorable rate, offering a balance between speed and optimization.⁵⁶
- **Custom:** Allows experienced treasury managers to set precise auction times and rate limits.⁵⁶

By migrating to the 1inch Fusion SDK for Node.js (npm install @1inch/fusion-sdk@2)⁵⁷, the corporate treasury achieves several third-order structural benefits. First, the treasury achieves complete gas abstraction. It no longer needs to maintain MATIC/POL gas balances in the Safe to fund automation scripts, drastically simplifying accounting overhead.⁵³ Second, it achieves infinite liquidity routing. If \$FARA liquidity migrates from Uniswap to SushiSwap, QuickSwap, or Aave, the resolvers automatically detect and route through the new depths without requiring the engineering team to rewrite the Node.js execution logic.⁴⁹ Finally, the 1inch Fusion protocol supports token plugins and custom ERC-20 routing.⁵⁸ As long as the resolvers identify a profitable arbitrage path, they will execute the settlement without requiring the treasury to deposit tokens into external farming or delegation contracts, preserving inherent security.⁵⁴

Mitigating MEV: The Polygon Private Mempool Integration

In public blockchain architectures, signed transactions are broadcast to an open memory pool (mempool) before being organized and included in a block by a validator.⁴⁶ This operational transparency created the systemic vulnerability known as Maximal Extractable Value (MEV), commonly referred to as the "Dark Forest."

Sophisticated algorithmic bots continuously monitor the public mempool for incoming large trades.⁶⁰ When a corporate TWAP buyback transaction is detected, the bot executes a "sandwich attack." It pays a higher gas fee to front-run the corporate purchase, artificially inflating the token price just milliseconds before the treasury's order executes. Immediately after the treasury buys at the inflated price, the bot back-runs the trade, dumping its tokens at a profit.⁶⁰ This predatory behavior forces the treasury to suffer maximum allowable slippage on every trade, silently siphoning billions of dollars of value out of the decentralized finance ecosystem.⁶⁰

While intent-based architectures like 1inch Fusion inherently protect the end-user (as the professional resolver bears the MEV risk during their on-chain settlement)⁵³, any fallback direct AMM interactions, module executions, or manual treasury interventions require robust, network-level protection. Historically, developers relied on specialized RPC endpoints like MEV Blocker or Flashbots Protect, which process millions of transactions monthly by routing them through secondary networks of searchers.⁴⁶ However, these solutions often introduce latency

and centralization trade-offs.⁴⁶

In April 2026, Polygon introduced a structural resolution to the MEV crisis on its PoS network: the Polygon Private Mempool.⁴⁶ Operating natively via a framework known as VeBloP, block production is handled by a known, validator-governed set of producers.⁴⁶ The Private Mempool allows developers to route transactions directly to these block producers, completely bypassing the public mempool visibility window.⁴⁶

Integrating this protection requires no complex infrastructure overhauls or new software dependencies. The Node.js execution environment simply segregates its operations:

- **Read Operations:** The ethers.js provider continues to query blockchain state, balances, and prices via standard public RPC URLs (e.g., Alchemy, Infura, or QuickNode) to ensure rapid data retrieval.⁶⁴
- **Write Operations:** All transaction submissions and signed payloads are routed exclusively through the new Private Mempool RPC endpoint ⁴⁶ (e.g., configuring Alchemy with the specific private endpoint syntax <https://polygon-mainnet.g.alchemy.com/v2/your-api-key>).⁶⁷

By implementing this single-line RPC modification ⁴⁶, the corporate buyback mechanism guarantees that its transactions remain totally invisible to predatory searchers until they are irrevocably confirmed in a block.⁴⁶ This structural property enforces a decentralization guarantee overseen by the validator set, ensuring transaction ordering remains unmanipulated and execution arrives exactly as submitted.⁴⁶

Autonomous Triggering Systems and Orchestration

The final architectural layer requires a decentralized, highly reliable engine to trigger the TWAP execution intervals on the Safe Zodiac module. Running a centralized cron job on a standard AWS EC2 instance introduces severe reliability risks; if the cloud server experiences downtime, the entire corporate buyback halts. The architecture must look to decentralized oracle networks for robust, 24/7 automation.

Evaluation of Decentralized Automation Networks

The two preeminent protocols for smart contract automation on Polygon are the Gelato Network and Chainlink Automation (formerly Keepers).

The Gelato Network

Gelato operates as a decentralized, multi-chain automation hub.⁷⁰ It utilizes a network of bots to execute smart contracts based on predefined conditions. To integrate, developers deploy a "resolver contract" containing a checker() function.⁷² If the checker() logic returns true (e.g., computationally verifying that precisely one hour has elapsed since the lastBought timestamp), Gelato's off-chain infrastructure executes the target function (e.g., buy()).⁷² Gelato is highly

effective for gasless, off-chain aware logic and broad EVM compatibility.⁷⁰

Chainlink Automation (Upkeeps)

Chainlink Automation v2.0 provides an industry-standard, trust-minimized solution backed by the same node operators securing trillions of dollars in decentralized transaction volume.⁷⁴

Chainlink utilizes the concept of an "Upkeep" to register jobs that the network will monitor.⁷⁷

The architecture supports two primary trigger mechanisms:

1. **Time-Based Triggers:** Scheduled execution utilizing standard cron expressions (e.g., triggering the buyback every day at exactly midnight UTC).⁷⁷
2. **Custom Logic Triggers:** The target smart contract explicitly implements the AutomationCompatibleInterface.⁷⁷ The decentralized off-chain Chainlink nodes continuously simulate the checkUpkeep function at zero gas cost to the user. If the logic detects that the USDC balance in the Safe exceeds the TWAP threshold and the mandatory time interval has elapsed, it returns true. The network then autonomously submits a transaction to call the performUpkeep function on-chain.⁷⁷

Chainlink Automation is the recommended solution due to its profound, native integration with the Polygon PoS ecosystem and unparalleled reliability guarantees.⁷⁶ By funding the Chainlink Upkeep contract with LINK tokens on Polygon⁸⁰, the corporate treasury completely outsources all DevOps monitoring tasks. The decentralized Chainlink nodes act as vigilant, highly reliable operators, ensuring the TWAP execution occurs with absolute precision, without ever relying on centralized web servers or vulnerable internal infrastructure.⁷⁶

Integrated System Data Flow and Execution Sequence

Synthesizing the evaluated technologies, the final architecture of the automated fiat-to-crypto buyback mechanism operates through a seamless, highly secure, and mathematically optimized data pipeline:

1. **Revenue Generation & Fiat Ingestion:** The Web2 travel platform generates variable monthly fiat revenue via Booking.com and Viator affiliate links, which is deposited into a traditional, regulated corporate bank account.
2. **API Detection & Programmatic Sweeping:** A lightweight, secure Node.js monitoring service detects the settled fiat. Utilizing the Plaid processor_token to bypass sensitive data storage¹³, the service calls the Circle Mint Core Functionality API (POST /v1/businessAccount/banks/wires)¹⁵ to initiate an immediate ACH pull, seamlessly converting the fiat into USDC at a 1:1 ratio.⁸
3. **On-Chain Payout & Bridging:** Upon receiving a programmatic webhook confirmation¹⁸ that the USDC has been successfully minted, the Circle Crypto Payouts API automatically transfers the native Polygon USDC directly to the corporate treasury's Gnosis Safe multisig on the Polygon PoS network, bypassing all third-party bridges.²⁰

4. **Decentralized Triggering:** Chainlink Automation nodes continuously monitor the Safe's balance via the zero-gas checkUpkeep simulation.⁷⁷ Upon detecting new liquidity and confirming the TWAP time interval has elapsed, Chainlink triggers the Safe's highly constrained custom StipendModule.²⁷
5. **Intent Generation & Settlement:** Instead of broadcasting a vulnerable market order to a specific AMM, the Safe Module utilizes the 1inch Fusion SDK ⁴⁷ to sign a zero-gas intent to swap USDC for \$FARA.⁵¹
6. **MEV-Protected Resolution:** Whitelisted 1inch resolvers, holding sufficient Unicorn Power, engage in a Dutch auction to find the most capital-efficient global routing for the intent.⁵² To ensure ultimate security, any direct on-chain state changes submitted by the automated backend or the Safe module utilize the new Polygon Private Mempool RPC, completely neutralizing all sandwich attack and front-running risks via the VeBloP architecture.⁴⁶
7. **Finality & Treasury Management:** The winning resolver executes the swap, paying all associated MATIC gas fees.⁵³ The acquired \$FARA tokens are routed directly to the corporate treasury's smart contract, where OpenZeppelin ERC20Burnable logic can be executed to permanently remove the tokens from the circulating supply ³², supporting the broader token ecosystem with optimal capital efficiency and zero manual intervention.

Works cited

1. Affiliate API | Lightweight Integration - Viator's Affiliate Program, accessed April 12, 2026, <https://partnerresources.viator.com/travel-commerce/affiliate/>
2. The Premier Travel Affiliate Program - by Viator, accessed April 12, 2026, <https://partnerresources.viator.com/>
3. How much will I receive in commission for bookings made from my referrals?, accessed April 12, 2026, <https://partnerhelp.viator.com/en/articles/69-how-much-will-i-receive-in-commission-for-bookings-made-from-my-referrals>
4. Commission and Payments - Booking.com Affiliate Partner Support, accessed April 12, 2026, <https://affiliates.support.booking.com/kb/s/article/Commission-and-Payments>
5. Automating TWAP Execution on Perpetual Protocol, accessed April 12, 2026, <https://blog.perp.fi/automating-twap-execution-on-perpetual-protocol-e96b1d9b4d1>
6. Crypto on-ramps: How they make fiat-to-crypto conversion secure and compliant - Stripe, accessed April 12, 2026, <https://stripe.com/resources/more/crypto-on-ramps>
7. Stripe fiat-to-crypto onramp, accessed April 12, 2026, <https://docs.stripe.com/crypto/onramp>
8. Core Functionality - Circle Docs, accessed April 12, 2026, <https://developers.circle.com/circle-mint/getting-started-with-the-circle-core-api>

9. Overview - Circle Docs, accessed April 12, 2026, <https://developers.circle.com/circle-mint/introducing-circle-mint>
10. Fiat Onramps: How Fiat-to-Crypto Conversion Works | Stripe, accessed April 12, 2026, <https://stripe.com/en-jp/resources/more/fiat-onramp-guide>
11. Circle Mint | Mint USDC, Unlock Business Efficiency, accessed April 12, 2026, <https://www.circle.com/circle-mint>
12. Fiat-to-Crypto Onramp Systems Explained - Stripe, accessed April 12, 2026, <https://stripe.com/resources/more/fiat-to-crypto-onramp-systems>
13. Auth - Astra | Plaid Docs, accessed April 12, 2026, <https://plaid.com/docs/auth/partnerships/astra/>
14. Circle Integrates With Plaid To Simplify ACH Payments - PR Newswire, accessed April 12, 2026, <https://www.prnewswire.com/news-releases/circle-integrates-with-plaid-to-simplify-ach-payments-301383753.html>
15. Quickstart: Deposit via Funds Transfer - Circle Docs, accessed April 12, 2026, <https://developers.circle.com/circle-mint/quickstart-deposit-via-funds-transfer>
16. ACH at Galileo - Introduction, accessed April 12, 2026, <https://docs.galileo-ft.com/pro/docs/ach-at-galileo>
17. Quickstart: Withdraw to Bank - Circle Docs, accessed April 12, 2026, <https://developers.circle.com/circle-mint/quickstart-withdraw-to-bank>
18. Manage Webhook Subscriptions - Circle Docs, accessed April 12, 2026, <https://developers.circle.com/circle-mint/webhook-subscription-management>
19. Enterprise Blockchain Payments Infrastructure | Polygon, accessed April 12, 2026, <https://polygon.technology/payments/enterprise>
20. USDC - Polygon Docs, accessed April 12, 2026, <https://docs.polygon.technology/pos/payments/transfers/transfer-usdc/>
21. Stablecoin News: Circle Opens USDC Payouts API to Circle Mint Singapore Partners, accessed April 12, 2026, <https://www.mexc.com/news/1014191>
22. Quickstart: Send a Stablecoin Payout - Circle Docs, accessed April 12, 2026, <https://developers.circle.com/circle-mint/send-stablecoin-payout>
23. Experience the power of USDC on Polygon PoS - Circle, accessed April 12, 2026, <https://www.circle.com/multi-chain-usdc/polygon>
24. USDC Polygon Guide: Contract Address, Wallet Setup, and How to Buy - MEXC Exchange, accessed April 12, 2026, <https://www.mexc.com/learn/article/usdc-polygon-guide-contract-address-wallet-setup-and-how-to-buy/1>
25. Multisig Best Practices to Maximize Transaction Security - Polygon, accessed April 12, 2026, <https://polygon.technology/blog/multisig-best-practices-to-maximize-transaction-security>
26. Breaking the vault: Lessons in custody security from the Bybit hack - Ocorian, accessed April 12, 2026, <https://www.ocorian.com/knowledge-hub/insights/breaking-vault-lessons-custody-security-bybit-hack>
27. How to Build a Gnosis Safe Module: Minimal Example in Solidity - DEV Community,

- accessed April 12, 2026,
<https://dev.to/doncesarts/how-to-build-a-gnosis-safe-module-minimal-example-in-solidity-5193>
28. Multisig Wallet Guide: Security, Setup & MPC Comparison 2026, accessed April 12, 2026,
<https://www.cobo.com/post/what-is-a-multisig-wallet-the-complete-guide-to-multi-signature-security>
 29. Multisig Wallet Transaction Models and Signature Rules - BitGo, accessed April 12, 2026, <https://www.bitgo.com/resources/blog/multisig-wallet-transactions/>
 30. How to use GnosisSafe Modules for secure on-chain Automation - YouTube, accessed April 12, 2026, <https://www.youtube.com/watch?v=vpnHR0rsMl0>
 31. Phala-Network/safe-vest-module: A Gnosis Safe module that vest tokens periodically · GitHub, accessed April 12, 2026,
<https://github.com/Phala-Network/safe-vest-module>
 32. How to Set Up a Token Buyback and Burn Program | ChainScore Guides, accessed April 12, 2026,
<https://chainscorelabs.com/guides/decentralized-applications-dapps-and-platforms/dapp-tokenomics-design/setting-up-a-token-buyback-and-burn-program>
 33. OpenZeppelin Contracts, accessed April 12, 2026,
<https://docs.openzeppelin.com/contracts>
 34. Contracts - OpenZeppelin Docs, accessed April 12, 2026,
<https://docs.openzeppelin.com/contracts/4.x>
 35. OpenZeppelin Defender, accessed April 12, 2026,
<https://defender.openzeppelin.com/>
 36. Defender | OpenZeppelin Docs, accessed April 12, 2026,
<https://docs.openzeppelin.com/defender>
 37. Compound Case Study: How Compound enhances DAO Operations with Defender - OpenZeppelin, accessed April 12, 2026,
<https://www.openzeppelin.com/news/compound-case-study-how-compound-enhances-dao-operations-with-defender>
 38. Execution Quality in Crypto: How to Measure Slippage, Liquidity, and Best Execution, accessed April 12, 2026,
<https://www.coinapi.io/blog/execution-quality-in-crypto>
 39. How to Swap Tokens on Uniswap with Ethers.js? - 101 Blockchains, accessed April 12, 2026, <https://101blockchains.com/swap-tokens-on-uniswap-with-ethers-js/>
 40. Swaps - Uniswap Docs, accessed April 12, 2026,
<https://docs.uniswap.org/concepts/protocol/swaps>
 41. Uniswap V3 SDK swap tutorial in TypeScript, Ethers.js - GitHub, accessed April 12, 2026, <https://github.com/TechGeorgii/uniswap-v3-sdk-tutorial-ts>
 42. Executing a trade in Node.js using UniSwap V3 protocol | by Vladimir Halme | Medium, accessed April 12, 2026,
<https://medium.com/@vhalme/executing-a-trade-in-node-js-using-uniswap-v3-protocol-9a0a51fe1360>
 43. Documentation (single page) - Ethers.js, accessed April 12, 2026,
<https://docs.ethers.org/v6/single-page/>

44. ethers.js, Swap on uniswapV3 failed tx - Stack Overflow, accessed April 12, 2026, <https://stackoverflow.com/questions/75124437/ethers-js-swap-on-uniswapv3-failed-tx>
45. Slippage Protection - Uniswap V3 Development Book, accessed April 12, 2026, https://uniswapv3book.com/milestone_3/slippage-protection.html
46. Polygon Launches Private Mempool: MEV Protection Is Now a One-Line Integration, accessed April 12, 2026, <https://polygon.technology/blog/polygon-launches-private-mempool-mev-protection-is-now-a-one-line-integration>
47. Native Token Swaps - 1inch Fusion SDK - Mintlify, accessed April 12, 2026, <https://www.mintlify.com/1inch/fusion-sdk/guides/native-token-swaps>
48. Perform an automated transaction using 1inch API - Ethereum Stack Exchange, accessed April 12, 2026, <https://ethereum.stackexchange.com/questions/89571/perform-an-automated-transaction-using-1inch-api>
49. Meet the 1inch Network's swap API for Polygon - 1inch Blog, accessed April 12, 2026, <https://blog.1inch.com/meet-the-1inch-networks-swap-api-for-polygon/>
50. Complete Web3 & DeFi API Suite | 1inch Developer Portal, accessed April 12, 2026, <https://business.1inch.com/portal/documentation>
51. How 1inch Fusion powers fast, MEV-protected swaps, accessed April 12, 2026, <https://blog.1inch.com/a-deep-dive-into-1inch-fusion/>
52. Modern DEXes, how they're made: 1inch Limit Order Protocol, Fusion & Fusion+ - MixBytes, accessed April 12, 2026, <https://mixbytes.io/blog/modern-dex-es-how-they-re-made-1inch-limit-order-protocols>
53. What is 1inch Fusion, and how does it work?, accessed April 12, 2026, <https://help.1inch.com/en/articles/6796085-what-is-1inch-fusion-and-how-does-it-work>
54. 1inch Fusion FAQ, accessed April 12, 2026, <https://help.1inch.com/en/articles/6800254-1inch-fusion-faq>
55. How the 1inch API integration powers Zerion Wallet's most efficient swaps, accessed April 12, 2026, <https://blog.1inch.com/how-the-1inch-api-integration-powers-zerion-wallets-most-efficient-swaps/>
56. A deep dive into 1inch Fusion - Medium, accessed April 12, 2026, <https://medium.com/1inch-network/a-deep-dive-into-1inch-fusion-03c50d16696Z>
57. 1inch/fusion-sdk · GitHub - GitHub, accessed April 12, 2026, <https://github.com/1inch/fusion-sdk>
58. 1inch Token Plugins: A Comprehensive Guide for Extending ERC20 Functionalities, accessed April 12, 2026, <https://gist.github.com/tradersnow222/818a7e9d92821587f98c3c6af386a219>
59. Fusion swap resolving: onchain component - 1inch Blog, accessed April 12, 2026, <https://blog.1inch.com/fusion-swap-resolving-onchain-component/>
60. MEV Blocker RPC Launched to Protect Users From Sandwich Attacks | COINCU

- on Binance Square, accessed April 12, 2026,
<https://www.binance.com/en/square/post/386631>
61. Polygon Launches Private Mempool, Enabling Streamlined Integration Of MEV Protection | MEXC News, accessed April 12, 2026,
<https://www.mexc.com/news/1000427>
 62. AndreiD/awesome-uniswap-v4-resources - GitHub, accessed April 12, 2026,
<https://github.com/AndreiD/awesome-uniswap-v4-resources>
 63. MEV Bots: how to protect yourself - CoW DAO, accessed April 12, 2026,
<https://cow.fi/learn/how-to-avoid-mev-bots>
 64. Polygon Private Mempool: What It Is and How to Use It - C# Corner, accessed April 12, 2026,
<https://www.c-sharpcorner.com/article/polygon-private-mempool-what-it-is-and-how-to-use-it/>
 65. Polygon Launches Private Mempool for One-Line MEV Protection | KuCoin, accessed April 12, 2026,
<https://www.kucoin.com/news/flash/polygon-launches-private-mempool-for-one-line-mev-protection>
 66. How To Get Pending Ethereum Mempool Transactions Using Ethers.js | QuickCodes, accessed April 12, 2026,
<https://www.youtube.com/watch?v=YjQj6uk9M98>
 67. How to Create a Private RPC Endpoint in 2025 - Alchemy, accessed April 12, 2026,
<https://www.alchemy.com/overviews/private-rpc-endpoint>
 68. Polygon Launches Private Mempool to Combat Frontrunning and Sandwich Attacks, accessed April 12, 2026,
<https://phemex.com/news/article/polygon-launches-private-mempool-to-combat-frontrunning-and-sandwich-attacks-70490>
 69. Did polygon just make mev protection default? : r/polygonnetwork - Reddit, accessed April 12, 2026,
https://www.reddit.com/r/polygonnetwork/comments/1scdl2c/did_polygon_just_make_mev_protection_default/
 70. Automate Your Smart Contract with Gelato - Developer DAO Blog, accessed April 12, 2026,
<https://blog.developerdao.com/automate-your-smart-contract-with-gelato>
 71. [Tutorial] How to Automate Your Smart Contracts Using Gelato Ops - YouTube, accessed April 12, 2026, <https://www.youtube.com/watch?v=YQGRT8aV5c>
 72. How to Automate Your Smart Contract Executions Using Gelato - DEV Community, accessed April 12, 2026,
<https://dev.to/gelato/how-to-automate-your-smart-contract-executions-1d14>
 73. How to Automate Your Smart Contract Executions Using Gelato, accessed April 12, 2026, <https://gelato.cloud/blog/automate-smart-contract-executions>
 74. How To Use Log Trigger Automation | Chainlink Engineering Tutorials - YouTube, accessed April 12, 2026, https://www.youtube.com/watch?v=nMFtqn timer pb8_k
 75. How To Create Self-Automating Smart Contracts | Chainlink Engineering Tutorials - YouTube, accessed April 12, 2026,
<https://www.youtube.com/watch?v=9uUk9neDqcM>

76. Chainlink Keepers Now Live on Polygon Mainnet to Automate Smart Contract DevOps, accessed April 12, 2026,
<https://polygon.technology/blog/chainlink-keepers-now-live-on-polygon-mainnet-to-automate-smart-contract-devops>
77. Introduction To Automation - Chainlink Fundamentals, accessed April 12, 2026,
<https://updraft.cyfrin.io/courses/chainlink-fundamentals/chainlink-automation/introduction-to-automation>
78. How To Create a Time-Based Upkeep With Chainlink Automation v1.2 - YouTube, accessed April 12, 2026, <https://www.youtube.com/watch?v=6GlvqggqHcLc>
79. How To Use Chainlink Automation To Trigger Smart Contracts in Solidity (NEW) - YouTube, accessed April 12, 2026,
<https://www.youtube.com/watch?v=dj0impNJdls>
80. Automate your Functions (Time-based Automation) | Chainlink Documentation, accessed April 12, 2026,
<https://docs.chain.link/chainlink-functions/tutorials/automate-functions>